

INF201 - Lecture 03

21st Sept 2026

Topics

- A comment about chained methods
- Unpacking lists and other containers
- File I/O
- Paths in Python

A comment about chained methods

```
In [24]: list_1 = [1, 2, 3]
list_2 = list_1.copy().append(4)
print(list_2)
```

None

- In Python you can chain multiple methods, as long as each method in the chain returns an object that supports the next method
- Since `list_1.copy()` returns a list, you can `append(4)` to that list
- So why is `list_2` `None`?
- This is because the `append` method modifies the list *in place* and returns `None`
- What happens here it therefore:
 - We get a (shallow) copy of `list_1`, but it is not given it's own name
 - To this copy, we append `4`
 - This list `[1, 2, 3, 4]` exist in memory for a split second
 - Python's garbage collection notice that the list is not assigned to any variable, and throws it out

```
In [25]: list_3 = list_1 + [4]
print(list_3)
list_4 = [*list_1, 4]
print(list_4)
print(list_1)
```

```
[1, 2, 3, 4]
[1, 2, 3, 4]
[1, 2, 3]
```

- `*` is an operator of unpacking lists or other iterables (such as tuples or sets)

```
In [26]: list_a = ["a", "b", "c"]
list_b = [1, 2]
print([list_a, list_b])
print(*list_a, *list_b)
```

[['a', 'b', 'c'], [1, 2]]
 ['a', 'b', 'c', 1, 2]

- `**` is a similar unpacking operator for mappings, such as dictionaries

```
In [27]: def print_dict(a, b):
print(f"a is {a} and b is {b}")

dict_a = {"a": 1, "b": 2}
print(dict_a)
print_dict(**dict_a)
dict_b = {"c": 3, "d": 4}
print(**dict_a, **dict_b)
```

{ 'a': 1, 'b': 2}
 a is 1 and b is 2
 { 'a': 1, 'b': 2, 'c': 3, 'd': 4}

- These operators are especially useful when calling functions

```
In [28]: def show_arguments(arg1, arg2, kwarg1=0, kwarg2=0):
print(f"1st: {arg1} | 2nd: {arg2} | 3rd: {kwarg1} | 4th: {kwarg2}")
return None

args = (1, 2) # Arguments
kwargs = {"kwarg1": 3, "kwarg2": 4} # Keyword arguments
show_arguments(*args, **kwargs) # Evaluates as: show_argument(1, 2, kwarg1=3, kwarg2=4)
```

1st: 1 | 2nd: 2 | 3rd: 3 | 4th: 4

File Input/Output - File I/O

- We will focus on files containing human-readable text
- There are Python packages for reading most file formats, also data stored in binary form (not human-readable)
- Today we will use csv-files and txt-files as examples
- We will discuss other common file formats next week

Reading from files

The very basics

To read data from a file we must:

1. Open the file
2. Read the content of the file
3. Close the file

Important points

- Python "talks" to operating system (OS) to get access to file.
- File may be located on a remote server, OS hides this from us/handles this for us.
- We must close the file when we are done.
- There is usually a limit on how many files we can have open simultaneously (~256).
- All open files will be closed when Python terminates.

An old-fashioned way to read from a file

```
In [29]: f = open("testfile_plain.txt")
text = f.read()
print(text)
f.close()
```

This is a simple text file.

It contains four lines, of which one is empty.
The file does not end with a new line character.

A more modern way to read from a file using contexts

```
In [30]: with open("testfile_plain.txt") as fc:
text = fc.read()

print(text)
```

This is a simple text file.

It contains four lines, of which one is empty.
The file does not end with a new line character.

- File is only open inside context.
- File will be closed properly no matter how we leave the context.
- Highly recommended for new code.
- `f` and `fc` are file handles, Python objects giving us access to the file.

```
In [31]: f
```

```
Out[31]: <_io.TextIOWrapper name='testfile_plain.txt' mode='r' encoding='cp1252'>
```

A more compact method

```
In [32]: text = open("testfile_plain.txt").read()  
text
```

```
Out[32]: 'This is a simple text file.\n\nIt contains four lines, of which one is empty.\nTh  
e file does not end with a new line character.'
```

```
In [33]: print(text)
```

This is a simple text file.

It contains four lines, of which one is empty.
The file does not end with a new line character.

- `open()` returns a file handle, we apply `read()` directly to it.
- We never store the file handle in a variable. Python garbage collection then automatically deletes the file handle object when the statement completes and closes the file in the process.
- When we display the text variable instead of `print()`ing it, we can see the newline characters `\n` explicitly.

Reading line-by-line

- In the above examples we read the entire content of the file
- What if the file is very large?

```
In [34]: with open("testfile_plain.txt") as tfile:  
        for line in tfile:  
            print(line)
```

This is a simple text file.

It contains four lines, of which one is empty.

The file does not end with a new line character.

- Why did this print extra lines?
- Reading one line at the time allows you to read from files that are too large to keep everything in memory.
- Also useful if you only need part of the file, for example information from a file header.

Line numbers

- `enumerate()` gives you access to the line number as well as the content
- This also works if you for example need the index when you iterate over a list

```
In [35]: with open("testfile_plain.txt") as tfile:  
        for line_number, line in enumerate(tfile):
```

```
print(f'{line_number:03d}: {line.strip()}')
```

000: This is a simple text file.

001:

002: It contains four lines, of which one is empty.

003: The file does not end with a new line character.

Comparison of simple file reading methods

- `for line in file:`
 - Reads one line at the time
 - $O(1)$ - constant memory: Memory requirement is not dependant on the file size
- `file.read()`
 - Reads the entire content of the file into a single string
 - $O(N)$ - linear memory: Memory requirement increases linearly with the size of the file
- `file.readlines()`
 - Reads the entire content of the file into a list with one element per line
 - $O(N)$ - linear memory: Memory requirement increases linearly with the size of the file

More advanced file readers

- Many Python packages have file readers for specific file types
- These will typically be wrappers around the built-in `open` which also handles context, and puts the data into an appropriate container
- Examples:
 - `read_csv` and `read_excel` from `pandas`
 - `csv.reader` from `csv`

Working with files and directories

- We will use `pathlib`, a modern library for working with files and folders
- Supports both Windows and Unix-like systems (such as Mac and Linux)

Terminology: POSIX

- Portable Operating System Interface
- IEEE standard for many features of operating systems, including file system
- Most Unix and Unix-like operating systems, including macOS, are fully or mostly POSIX compliant
- <https://no.wikipedia.org/wiki/POSIX>

What is a path?

- A way (path) to a directory or file on our system
- Each path has
 - a starting point
 - steps (intermediate directories)
 - a destination (directory or file)
- Absolute paths
 - start from the root of the file system hierarchy or drive (Windows)
 - POSIX: start with `/`
 - Windows
 - start with drive letter, e.g., `C:`
 - start with `\\`
 - point to a uniquely defined place in the file system
- Relative paths
 - start from the directory "where we are": *current working directory* (cwd)
 - start with any letter other than `/` (Posix) or `\\` (Windows)
 - starting point determines which file or directory a relative path points to
- Special paths
 - `.` is the current directory
 - `..` is the parent of the current directory
 - `~` is the user's home directory (Posix only)

pathlib

- <https://docs.python.org/3/library/pathlib.html>

```
In [36]: from pathlib import Path

# Look at the current working directory
cwd = Path.cwd()
cwd
```

```
Out[36]: WindowsPath('c:/Users/tutor5923/OneDrive - Norwegian University of Life Sciences/inf201/h26/repo_for_students/lectures/lecture_03')
```

```
In [37]: # Check if directory exists
cwd.exists()
```

```
Out[37]: True
```

```
In [38]: # Check if this is a directory
cwd.is_dir()
```

```
Out[38]: True
```

```
In [39]: # Check if this is a file
cwd.is_file()
```

Out[39]: False

```
In [40]: # Check if this is an absolute path
cwd.is_absolute()
```

Out[40]: True

```
In [41]: # Get path directly above cwd (it's parent directory)
cwd.parent
```

Out[41]: WindowsPath('c:/Users/tutor5923/OneDrive - Norwegian University of Life Sciences/inf201/h26/repo_for_students/lectures')

```
In [42]: # List all parents
list(cwd.parents)
```

Out[42]: [WindowsPath('c:/Users/tutor5923/OneDrive - Norwegian University of Life Sciences/inf201/h26/repo_for_students/lectures'),
WindowsPath('c:/Users/tutor5923/OneDrive - Norwegian University of Life Sciences/inf201/h26/repo_for_students'),
WindowsPath('c:/Users/tutor5923/OneDrive - Norwegian University of Life Sciences/inf201/h26'),
WindowsPath('c:/Users/tutor5923/OneDrive - Norwegian University of Life Sciences/inf201'),
WindowsPath('c:/Users/tutor5923/OneDrive - Norwegian University of Life Sciences'),
WindowsPath('c:/Users/tutor5923'),
WindowsPath('c:/Users'),
WindowsPath('c:/')]

```
In [43]: # List the parts that make up the path
cwd.parts
```

Out[43]: ('c:\\',
'Users',
'tutor5923',
'OneDrive - Norwegian University of Life Sciences',
'inf201',
'h26',
'repo_for_students',
'lectures',
'lecture_03')

```
In [44]: # Create a relative path
relpath = Path('.')
relpath
```

Out[44]: WindowsPath('.')

```
In [45]: relpath.is_absolute()
```

Out[45]: False

```
In [46]: # Resolve this to a relative path
```

```
relpath.resolve()
```

```
Out[46]: WindowsPath('C:/Users/tutor5923/OneDrive - Norwegian University of Life Sciences/inf201/h26/repo_for_students/lectures/lecture_03')
```

Building paths

- Use `pathlib` functionality, not string addition
- Avoids issues with different paths in Windows and Unix

```
In [47]: # Construct a path with / (works on any system)
lec02 = Path("../") / "lecture_02"
lec02
```

```
Out[47]: WindowsPath('../lecture_02')
```

```
In [48]: lec02.resolve()
```

```
Out[48]: WindowsPath('C:/Users/tutor5923/OneDrive - Norwegian University of Life Sciences/inf201/h26/repo_for_students/lectures/lecture_02')
```

```
In [49]: # Can also call Path
lec01 = Path("../", "lecture_01")
lec01
```

```
Out[49]: WindowsPath('../lecture_01')
```

```
In [50]: lec01.resolve()
```

```
Out[50]: WindowsPath('C:/Users/tutor5923/OneDrive - Norwegian University of Life Sciences/inf201/h26/repo_for_students/lectures/lecture_01')
```

```
In [51]: # Note that resolve() does not change lec01 in place
lec01
```

```
Out[51]: WindowsPath('../lecture_01')
```

Creating directories and moving files around

- Let's create some directories and files for demonstration

```
In [52]: demo_dir = Path("demo")
```

```
In [53]: demo_dir.exists()
```

```
Out[53]: False
```

```
In [54]: demo_dir.mkdir()
```

```
In [55]: for n in range(4):
          subdir = demo_dir / str(n)
          subdir.mkdir()
```

```

for filenum in range(3):
    cname = chr(97 + filenum)
    fname = subdir / (cname + ".txt")
    with open(fname, "w") as f:
        f.write(f"Customer nr: {n}\nCustomer name: {cname}\nPayload\n")

```

```
In [56]: print(open("demo/0/a.txt").read())
```

```

Customer nr: 0
Customer name: a
Payload

```

- These files are now stored in a hierarchical structure
- Let's make a new flat directory for all the files

```

In [57]: new_dir = Path("demo_new")
         new_dir.mkdir()

         for old_name in demo_dir.glob("*/*.txt"):
             with old_name.open() as f:
                 c_num = int(f.readline().split(":")[-1].strip())
                 c_name = f.readline().split(":")[-1].strip()
                 new_name = new_dir / f"invoice_{c_name:s}_{c_num:03d}.dat"
                 old_name.rename(new_name)

```

```
In [58]: list(new_dir.glob('**/*'))
```

```

Out[58]: [WindowsPath('demo_new/invoice_a_000.dat'),
          WindowsPath('demo_new/invoice_a_001.dat'),
          WindowsPath('demo_new/invoice_a_002.dat'),
          WindowsPath('demo_new/invoice_a_003.dat'),
          WindowsPath('demo_new/invoice_b_000.dat'),
          WindowsPath('demo_new/invoice_b_001.dat'),
          WindowsPath('demo_new/invoice_b_002.dat'),
          WindowsPath('demo_new/invoice_b_003.dat'),
          WindowsPath('demo_new/invoice_c_000.dat'),
          WindowsPath('demo_new/invoice_c_001.dat'),
          WindowsPath('demo_new/invoice_c_002.dat'),
          WindowsPath('demo_new/invoice_c_003.dat')]

```

```
In [59]: print(open("demo_new/invoice_b_002.dat").read())
```

```

Customer nr: 2
Customer name: b
Payload

```

```
In [60]: list(demo_dir.glob('**/*'))
```

```

Out[60]: [WindowsPath('demo/0'),
          WindowsPath('demo/1'),
          WindowsPath('demo/2'),
          WindowsPath('demo/3')]

```

- All files have been moved from the old to the new directory
- What happens if we try to create the same directory again?

In [61]: `demo_dir.mkdir()`

```
-----
FileExistsError                                Traceback (most recent call last)
Cell In[61], line 1
----> 1 demo_dir.mkdir()

File ~\AppData\Local\Python\pythoncore-3.14-64\Lib\pathlib\__init__.py:1011, in Path.mkdir(self, mode, parents, exist_ok)
    1007 """
    1008 Create a new directory at this given path.
    1009 """
    1010 try:
-> 1011     os.mkdir(self, mode)
    1012 except FileNotFoundError:
    1013     if not parents or self.parent == self:

FileExistsError: [WinError 183] Cannot create a file when that file already exists:
'demo'
```

- If we want our script to overwrite the existing directory, we need to flag this specifically
- Note: this does not delete files or subdirectories in the existing directory

In [62]: `demo_dir.mkdir(exist_ok=True)`

- Often we also include the argument `parents=True`, which ensures that any missing parent directories will also be created

In [63]: `another_demo_dir = Path("demo_again", "sub_dir", "sub_sub_dir")`
`another_demo_dir.mkdir(parents=True, exist_ok=True)`

Globbing: looking at files and directories

- "Globbing" means to find all files or directories matching a pattern
- `*` is a wildcard matching anything, `?` matches any single letter
- `Path.glob()` returns a generator, we need to convert explicitly to a list or iterate

In [64]: `list(lec02.glob("*.py"))`

Out[64]: `[WindowsPath('../lecture_02/aliasing_example.py'),`
`WindowsPath('../lecture_02/f_string_examples.py'),`
`WindowsPath('../lecture_02/list_comprehension.py'),`
`WindowsPath('../lecture_02/list_copying.py'),`
`WindowsPath('../lecture_02/mutability.py'),`
`WindowsPath('../lecture_02/type_hint_functions.py')]`

```
In [65]: for item in Path('..').glob('*'):
         if item.is_file():
             print('FILE: ', end='')
         elif item.is_dir():
             print('DIR : ', end='')
         else:
             print('????: ', end='')
         print(item)
```

DIR : ..\lecture_01

DIR : ..\lecture_02

DIR : ..\lecture_03